

Agent And On-Chain Event Map

Audience: Brain engineering, protocol, platform, and on-call owners

Last updated: 2026-06-21

Code review target: `review-brain-core-codebase` at `386e8da`

This document closes the gap between the agent docs, MCP docs, audit docs, and contract docs. It names the agent surfaces, the off-chain events they emit, the on-chain events the system relies on, and the production reconciliation behavior that must exist before mainnet autonomy is credible.

Summary

Yes, Brain needs dedicated documentation for agents and on-chain events. Existing docs cover the parts separately:

- `protocol/agents.md` and `protocol/agent-contributions.md` describe agent concepts and provenance.
- `docs/agent-autonomy-v3.md` and `docs/external-agent-onboarding.md` describe autonomy and onboarding.
- `docs/mcp-architecture.md`, `mcp-server/mcp-authentication.md`, and `docs/adr/0002-mcp-propose-not-execute.md` describe the MCP boundary.
- `smart-contracts/*.md` and `docs/adr/0006-onchain-audit-anchoring.md` describe contract behavior and audit anchoring.

What was missing is a single operational map for developers: when an agent acts, which off-chain row or audit event is produced, which on-chain event matters, which process consumes it, and what happens during replay, reorg, or incident response.

Agent Runtime Surfaces

Surface	Code anchor	Production stance
Internal business agents	<code>services/internal-agents/src/*</code> and <code>services/agent-router/src/*</code>	Agents propose actions and persist run history. Financial proposals go through <code>PaymentIntent</code> and the pre-execution gate.
MCP external agents	<code>services/mcp/src/*</code>	Read and propose only. <code>payment_intent.execute</code> is deliberately not exposed through MCP.
Raw contribution agents	<code>services/mcp/src/tools/raw.ts</code> , <code>protocol/agent-contributions.md</code>	Agent-supplied raw evidence is low-trust, provenance-capped, and subject to quarantine/corroboration before it can influence automation.
On-chain registered agents	<code>contracts/src/BrainMCPAgentRegistry.sol</code>	Third-party agents are registered with tenant scope hash and behavior hash. Runtime auth must match the on-chain registration.

Surface	Code anchor	Production stance
On-chain session-key executor	<code>contracts/src/BrainSmartAccount.sol</code>	Session keys are scoped by policy version, target allowlist, selector allowlist, amount caps, period caps, nonce, and pause state.

Agent Event And State Flow

The normal financial-agent flow is:

1. Agent produces a proposal.
2. Agent router records the run and terminal state in `agent_runs` and related routing/run-history projections.
3. Financial proposals become `ledger_payment_intents`.
4. Approval and execution flow through the Execution service.
5. The deterministic pre-execution gate runs before dispatch.
6. The outbox claims and dispatches the rail action.
7. Receipts, reservations, audit events, and PaymentIntent status are updated from the rail outcome.

The MCP flow adds an outer audit envelope:

1. HTTP auth validates the principal.
2. MCP auth verifies the agent record, tenant, scope, and scope hash.
3. `tools/call` enforces the tool scope.
4. The server emits `agent.mcp.tool_called` for both accepted and rejected tool calls.
5. Mutating tools emit their own domain events through the service they call.

Important invariant: MCP is not an execution rail. It can propose, cancel, list, read memory, read proofs, and contribute raw evidence. It must not execute money movement directly.

On-Chain Event Catalog

These are the contract events that should be listed in runbooks, dashboards, and release evidence.

Contract	Event	Meaning	Operational use
<code>BrainMCPAgentRegistry</code>	<code>AgentRegistered</code>	Tenant authorized an external MCP agent with scope hash and behavior hash.	Agent onboarding evidence, scope drift checks, auth verifier cache validation.
<code>BrainMCPAgentRegistry</code>	<code>AgentRevoked</code>	Tenant revoked an external agent.	Incident response, access disablement, cache invalidation, SIEM alert.
<code>BrainMCPAgentRegistry</code>	<code>AgentBehaviorUpdated</code>	Tenant updated the registered behavior hash.	Runtime behavior-hash gate evidence and deployment-change tracking.

Contract	Event	Meaning	Operational use
BrainMCPAgentRegistry	TenantSignerSet	Tenant signer added or removed.	Admin audit, onboarding/offboarding evidence.
BrainPolicyRegistry	PolicyRegistered	Tenant policy hash/version registered on-chain.	Policy provenance, policy-version binding for execution.
BrainPolicyRegistry	TenantSignerSet	Policy signer added or removed.	Admin audit and policy-control evidence.
BrainSmartAccount	SessionKeyGranted	Root owner granted a scoped session key.	Rail enablement evidence, key inventory.
BrainSmartAccount	SessionKeyRevoked	Session key permanently revoked.	Incident response, rail disablement proof.
BrainSmartAccount	SessionKeyPaused / SessionKeyResumed	Session key kill switch toggled.	Operational halt/resume evidence.
BrainSmartAccount	AccountPaused / AccountResumed	Account-wide kill switch toggled.	Emergency stop evidence.
BrainSmartAccount	OwnershipTransferStarted / OwnershipTransferred	Root owner rotation started/completed.	Custody and key-management audit.
BrainSmartAccount	AgentActionExecuted	Session key executed an allowed action with tenant, agent, policy version, target, selector, amount, and calldata hash.	On-chain receipt correlation, nonce replay proof, rail reconciliation.
BrainAuditAnchor	AnchorPublished	Per-tenant audit Merkle root published.	Tamper-evidence proof and audit verifier evidence.
BrainAuditAnchor	PublisherTransferStarted / PublisherChanged	Anchor publisher rotation started/completed.	Anchoring operations and custody audit.
IBrainEscrow	EscrowLocked	x402/M2M escrow funded.	Escrow state binding and payment lifecycle evidence.
IBrainEscrow	EscrowReleased	Escrow released to payee, possibly partially.	Settlement proof and dispute/milestone evidence.
IBrainEscrow	EscrowRefunded	Escrow refunded to payer, possibly partially.	Refund/dispute evidence.
IBrainReputationRegistry	ReputationPublished	Agent reputation pointer root published.	Policy input only. It must not replace deterministic money gates.
IBrainReputationRegistry	AttestorChanged	Reputation attester rotated.	Reputation-source custody audit.

Event Consumption Rules

Use these rules for production services and runbooks:

- Treat the off-chain ledger/audit tables as the application source of truth for Brain state transitions.
- Treat on-chain events as settlement, custody, registration, and tamper-evidence proof that must be reconciled to off-chain state.
- Never rely on an event watcher as an unguarded money executor.
- Every chain event consumer must be idempotent by `(chain_id, contract_address, tx_hash, log_index)`.
- Every watcher must store a durable cursor per chain and contract.
- Confirmations must be profile-specific. Staging may use a short confirmation window; mainnet must use an explicit finality policy.
- Reorg handling must roll back or mark affected derived rows as uncertain until the replacement canonical logs are processed.
- Replay must be supported from a known block range without duplicating effects.
- Unknown or malformed events go to dead letter with contract, topic, tx hash, block number, and decode error.
- Dashboards must show watcher lag, last processed block, confirmation depth, decode failures, dead-letter count, replay count, and correlation failures.

Correlation Keys

Flow	Primary off-chain key	On-chain key	Required correlation
Audit anchoring	Audit window id, tenant id, Merkle root	<code>AnchorPublished(tenantId, root, period*)</code>	Root and event count must match the published proof bundle.
MCP agent registration	Agent id, tenant id, scope hash, behavior hash	<code>AgentRegistered(agentId, tenantId, scopeHash, behaviorHash)</code>	Runtime auth verifier must reject drift between JWT/off-chain record and registry state.
Payment rail execution	PaymentIntent id, outbox id, receipt id, reservation id	<code>AgentActionExecuted(tenantId, agentId, policyVersion, target, selector, amount, calldataHash)</code>	Receipt must match target, selector, value/amount, calldata hash, nonce, and transaction hash.
Escrow lock/release/refund	PaymentIntent id, escrow id, job terms hash	<code>EscrowLocked, EscrowReleased, EscrowRefunded</code>	Escrow id, token, amount, payee/payer, terms hash, and settlement totals must match expected state.
Policy registration	Tenant id, policy version, policy hash	<code>PolicyRegistered(tenantId, version, policyHash)</code>	Execution policy version must be traceable to registered policy hash.
Reputation update	Agent id, score root, epoch	<code>ReputationPublished(agentId, scoreRoot, epoch)</code>	Reputation may tighten policy decisions, but cannot authorize a money movement.

Production Gaps To Track

These are documentation and implementation items that should remain visible in readiness evidence until closed:

1. Add an on-chain event ingestion runbook with cursor schema, finality policy, reorg handling, dead-letter handling, and replay procedure.
2. Add an agent incident runbook that covers MCP revocation, session-key pause, smart-account pause, quarantine, proof export, and audit replay.
3. Add dashboards for agent scope drift, rejected MCP calls, agent proposal volume, raw contribution quarantine, on-chain watcher lag, and receipt correlation failures.
4. Add release evidence that Base Sepolia exercises `SessionKeyGranted`, `AgentActionExecuted`, and audit-anchor verification from a real transaction.
5. Keep mainnet escrow blocked until external audit, bytecode verification, and operator attestation are complete.

Diagrams

- `docs/diligence/diagrams/agent-onchain-event-map.mmd`
- `docs/diligence/diagrams/onchain-event-reconciliation-lifecycle.mmd`